FindIt — Code Comments Guide

Guidance for documenting classes, methods, and business logic

1. Purpose

Code comments should explain the purpose of a class, method, or non-obvious decision. They should clarify why code exists rather than repeat each line.

2. Main Application Class

Example comment:
`// Starts the Spring Boot application.`

3. Controller

Example comment:
`// Handles web requests and returns the appropriate response.`
Add comments for important request mappings, form validation, and redirects or view selection.

4. Entity

Example comment:
`// Represents an item persisted in the database.`
Document the purpose of important fields and any non-obvious validation or relationships.

5. Service

Example comment:
`// Contains business logic for item-related operations.`
Describe important business rules, status changes, or validation decisions.

6. Repository

Example comment:
`// Provides data-access operations for Item entities.`
Explain custom query methods where their purpose is not obvious.

7. Good Commenting Practices

• Use clear, concise language.
• Keep comments accurate when code changes.
• Explain non-obvious logic and important assumptions.
• Avoid redundant comments such as 'increment i' directly above i++.
• Never put passwords, tokens, or private data in comments.

8. Important Note

The examples are comment-writing guidance, not replacement source code. Add comments to the actual project classes and methods, and do not create empty placeholder classes solely to match this document.